DNA-DB vs MySQL vs PostgreSQL

DNA-DB v3.5.0 · MySQL 8.4.6 · PostgreSQL 17.11

Итоги сравнительного бенчмарка трёх систем управления базами данных: пропускная способность по семи фазам, задержки p50/p95/p99, масштабирование записи и соответствие SQL по 32 проверкам. Все системы — на настройках долговечности по умолчанию; каждое чтение проверено на корректность.

Дата прогона: 31 октября 2026 года
Платформа: Linux x86_64 · 2 vCPU · 4 ГБ RAM · durability по умолчанию

Содержание

1. Резюме 2

2. Объекты сравнения 2

3. Методика бенчмарка 4

4. Сводная таблица сравнения 5

5. Пропускная способность: графики и пояснения 5

6. Задержки: p50, p95, p99 7

7. Пиковая запись и масштабирование 9

8. Соответствие SQL: 32 проверки 10

9. Почему цифры именно такие 11

10. Выводы и рекомендации 12

1. Резюме

Настоящий отчёт подводит итоги сравнительного бенчмарка трёх систем управления базами данных: молодого движка **DNA-DB Core v3.5.0** (Rust, LSM-архитектура, заточен под геномные нагрузки), **MySQL 8.4.6** (InnoDB) и **PostgreSQL 17.11**. Все три системы работали на одной Linux-машине (x86_64, 2 vCPU, 4 ГБ RAM) 31 октября 2026 года, получали идентичную нагрузку, одинаковые ключи и одну последовательность случайных чисел, а главное — использовали **настройки долговечности по умолчанию**: strict fsync у DNA-DB, innodb_flush_log_at_trx_commit=1 у MySQL, synchronous_commit=on у PostgreSQL. Ни одна система не тюнинговалась, кэш между фазами не переиспользовался, и каждая фаза начиналась с одного и того же состояния данных.

Результат измерялся, а не предполагался: каждое из 4 000 KV-чтений и 1 500 SQL-запросов сверялось с эталонным значением, записанным ранее. Все три системы ответили корректно — **0 расхождений и 0 ошибок**. По скорости картина следующая: DNA-DB выигрывает пять фаз из семи, PostgreSQL — две (многопоточная и однопоточная SQL-нагрузка на 5–25 %), MySQL не выигрывает ни одной. Самые показательные разрывы: однопоточная запись — **×8,3** к MySQL и **×1,9** к PostgreSQL, точечное чтение — **×4,0** и **×2,5** соответственно; в пакетном режиме DNA-DB достигает **298 936 строк/с** — максимального значения во всём прогоне.

36 260	43 779	298 936	0
строк/с — одиночная запись (×8,3 к MySQL)	чтений/с — точечный поиск по ключу	строк/с — пакетная загрузка	расхождений в 5 500 проверенных чтениях

Скорость — не единственный критерий. По покрытию SQL PostgreSQL реализует все 32 проверки эталонного теста, MySQL — 31 из 32, DNA-DB — 25 из 32; однако принципиально, что всё, что DNA-DB принимает, возвращает эталонно верные строки (0 неверных результатов у всех трёх систем). Отставание DNA-DB — это покрытие фич (RIGHT/FULL OUTER JOIN, UNION, явные транзакции), а не корректность. Одновременно DNA-DB предлагает то, чего нет у соперников: язык DnaQL для нечёткого поиска последовательностей, 2-битную упаковку нуклеотидов (0,25 байта на основание) и самовосстановление после порчи секторов по кодам Рида — Соломона. Итоговый выбор по сценариям приведён в главе 10.

2. Объекты сравнения

В сравнении участвуют три системы разных классов, и это важно понимать при чтении цифр. MySQL и PostgreSQL — зрелые универсальные клиент-серверные СУБД с тридцатилетней историей, полным SQL, MVCC, репликацией и огромной экосистемой. DNA-DB Core — компактный движок (около 25 тысяч строк на Rust), который одновременно является встраиваемой библиотекой и TCP-сервером, и который сознательно оптимизирован под узкую нишу: долговечные одноключевые операции, геномные данные и выживание при аппаратных сбоях.

2.1. DNA-DB Core v3.5.0

DNA-DB построен по классической LSM-схеме: журнал упреждения (WAL) с CRC32 на каждый кадр, memtable на skip-листе, SSTable-сегменты с bloom-фильтром и разреженным индексом, фоновая компакция. Ключевая фишка — «хромосомная» модель данных: ключи вида `chr_users:14`, 2-битная упаковка нуклеотидов (четыре основания в одном байте), язык DnaQL с нечётким поиском по расстоянию Хэмминга и Левенштейна и био-плагинами (Smith-Waterman, поиск ORF, GC-окна). Релиз v3.5.0 закрывает все критические находки аудита v3.4.0: случайный bootstrap-пароль вместо дефолтного, ограничение глубины парсера (128 уровней против stack overflow), rollback WAL при сбое зеркала, честная изоляция Serializable и бесfсинхронное чтение.

Хранение защищено двумя слоями: зеркалирование «двух нитей» (Strand A/B — аналог RAID-1 с горячей заменой) и помехоустойчивое кодирование Рида — Соломона GF(2^8) внутри каждого сегмента (k=4 данных + m=2 чётности, блоки 4 КиБ). При чтении битого блока полоса восстанавливается из уцелевших шардов и перезаписывается — это и есть «самовосстановление». Безопасность: rustls TLS 1.2/1.3 на обоих слушателях, Argon2id-хэши паролей, блокировка перебора по IP и имени пользователя, XChaCha20-Poly1305 шифрование значений на диске, по-табличные ACL и веб-панель администрирования.

2.2. MySQL 8.4.6 и PostgreSQL 17.11

MySQL работал с движком InnoDB и настройкой innodb_flush_log_at_trx_commit=1 — это долговечность по умолчанию: каждый коммит оплачивает fsync журнала, а doublewrite-буфер защищает страницы таблиц от разрыва записи. PostgreSQL 17.11 запущен с synchronous_commit=on — той же гарантией: WAL сбрасывается на диск перед подтверждением клиента, а после первой модификации страницы в окне чекпойнта в журнал дописывается её полный образ (full-page image). Оба сервера — официальные сборки без тюнинга, что соответствует духу сравнения: не «кто лучше настроен», а «что получает разработчик сразу после установки».

Параметр	DNA-DB v3.5.0	MySQL 8.4.6	PostgreSQL 17.11
Язык реализации	Rust	C / C++	C
Модель хранения	LSM-дерево (WAL + memtable + SSTable)	Страничное B-дерево (InnoDB)	Страничное heap-хранение
SQL	Подмножество (25/32 теста) + DnaQL	Полный	Полный
Долговечность	strict: fsync WAL на каждое подтверждение	innodb_flush_log_at_trx_commit=1	synchronous_commit=on
Шифрование на диске	XChaCha20-Poly1305 (AEAD на значение)	Опционально (tablespace)	Опционально (pgcrypto/TDE в форках)
Самовосстановление	Рид — Соломон RS(4+2) + зеркало Strand A/B	Репликация (отдельный механизм)	Репликация (отдельный механизм)

Таблица 1. Краткий профиль трёх систем

3. Методика бенчмарка

Бенчмарк выполнялся скриптом `scripts/bench_compare.py` из состава репозитория: один клиентский процесс Python 3.12 подключался к трём серверам по очереди и прогонял семь фаз с идентичными параметрами. Датасет детерминирован: ключи `chr1:0000000…chr1:0099999`, значение 160 байт, порядок операций перемешивается генератором с фиксированным зерном (42 для загрузки, 777 для смешанной фазы) — то есть все три системы получили строго одинаковую работу. Клиент к DNA-DB ходит по собственному бинарному TCP-протоколу поверх TLS, к MySQL — через PyMySQL, к PostgreSQL — через psycopg2; все драйверы в режиме autocommit.

Каждая одиночная операция — отдельная долговечная транзакция: одна запись WAL с fsync на каждое подтверждение во всех трёх системах. Именно fsync определяет абсолютные значения ops/s на записывающих фазах, поэтому сравнение честное: никто не экономит на сбросе на диск. Корректность не предполагается, а измеряется — каждый прочитанный байт сравнивается с эталоном, после многопоточной и пакетной фаз сверяется итоговое число строк (8 000/8 000 на всех системах).

№	Фаза	Объём	Что измеряет
1	Загрузка KV, один поток	10 000 записей	Цену одной долговечной записи (put + fsync + ack)
2	Многопоточная загрузка	8 потоков × 1 000 INSERT	Конкурентную запись одиночных строк
3	Пакетная загрузка	8 000 строк, 40 INSERT по 200	Пиковый поток записи с батчингом
4	Точечное чтение	4 000 операций	Поиск по первичному ключу (bloom + индекс)
5	Смешанная 80/20	8 потоков × 500 операций	Реалистичный профиль чтение/запись
6	SQL INSERT	3 000 строк	SQL-путь записи (парсер + планировщик + WAL)
7	SQL SELECT по PK	1 500 запросов	SQL-путь чтения с планом по индексу

Таблица 2. Фазы бенчмарка (одинаковая нагрузка и RNG для всех систем)

Честные оговорки, которые важно прочитать перед цитированием цифр. Во-первых, это **микробенчмарк одноключевых операций**: он ничего не говорит о сложных JOIN, аналитике и репликации — там MySQL и PostgreSQL сегодня сильнее. Во-вторых, это **один прогон**: каждая фаза длится 2–7 секунд, и разброс между повторами составляет несколько процентов. В-третьих, конфигурации серверов — заводские; целью было измерить «коробочную» производительность, а не искусственно раздутый рекорд. Наконец, среда — общая виртуальная машина с 2 vCPU, поэтому абсолютные значения ниже, чем на выделенном железе, но относительный порядок систем устойчив.

4. Сводная таблица сравнения

Главная таблица отчёта собирает пропускную способность всех семи фаз. Жирным выделено лучшее значение фазы; правая колонка объясняет, во сколько раз лидер опережает преследователей. Формат «ops/s» на фазах 1–5 — операции в секунду, на фазах 2, 3 и 6 — строк в секунду; единица везде одна внутри фазы, поэтому отношение корректно.

Фаза	DNA-DB v3.5.0	MySQL 8.4.6	PostgreSQL 17.11	Кто выигрывает и насколько
Загрузка KV (1 поток)	**36 260**	4 353	19 182	DNA-DB: ×8,3 к MySQL, ×1,9 к PG
Многопоточная загрузка	17 636	8 316	**22 157**	PostgreSQL: ×1,26 к DNA-DB, ×2,7 к MySQL
Пакетная загрузка	**298 936**	162 166	215 926	DNA-DB: ×1,8 к MySQL, ×1,4 к PG
Точечное чтение	**43 779**	11 049	17 565	DNA-DB: ×4,0 к MySQL, ×2,5 к PG
Смешанная 80/20	**20 411**	8 403	17 985	DNA-DB: ×2,4 к MySQL, ×1,1 к PG
SQL INSERT	**24 986**	4 003	16 811	DNA-DB: ×6,2 к MySQL, ×1,5 к PG
SQL SELECT по PK	22 470	8 916	**23 638**	PostgreSQL: ×1,05 к DNA-DB, ×2,7 к MySQL
Проверенных чтений с неверным значением	0	0	0	Корректность у всех
Сверено строк после MT/пакетной загрузки	8 000/8 000	8 000/8 000	8 000/8 000	Полное совпадение

Таблица 3. Сводный результат бенчмарка (ops/s либо rows/s, больше — лучше)

Из семи фаз DNA-DB выигрывает пять, причём четыре из них — с ощутимым отрывом (от ×1,4 до ×8,3 к ближайшему сопернику). PostgreSQL забирает две фазы, связанные с конкурентной и SQL-записью: в многопоточной вставке он быстрее DNA-DB в 1,26 раза, в однопоточном SQL SELECT — в 1,05 раза (паритет). MySQL ни в одной фазе не оказывается первым, но на конкурентных нагрузках его отставание от лидера невелико (×1,1–×2,6), тогда как на одиночных долговечных операциях разрыв достигает ×6–×8, что объясняется ценой его конвейера коммита и doublewrite-буфера (подробно в главе 9).

5. Пропускная способность: графики и пояснения

Рисунок 1 показывает все семь фаз в логарифмической шкале: диапазон значений слишком широк (от 4 003 до 298 936), чтобы линейная ось сохранила читаемость мелких столбцов. Точные значения подписаны у каждого столбца, поэтому логарифм здесь — только визуальный приём, а не искажение.

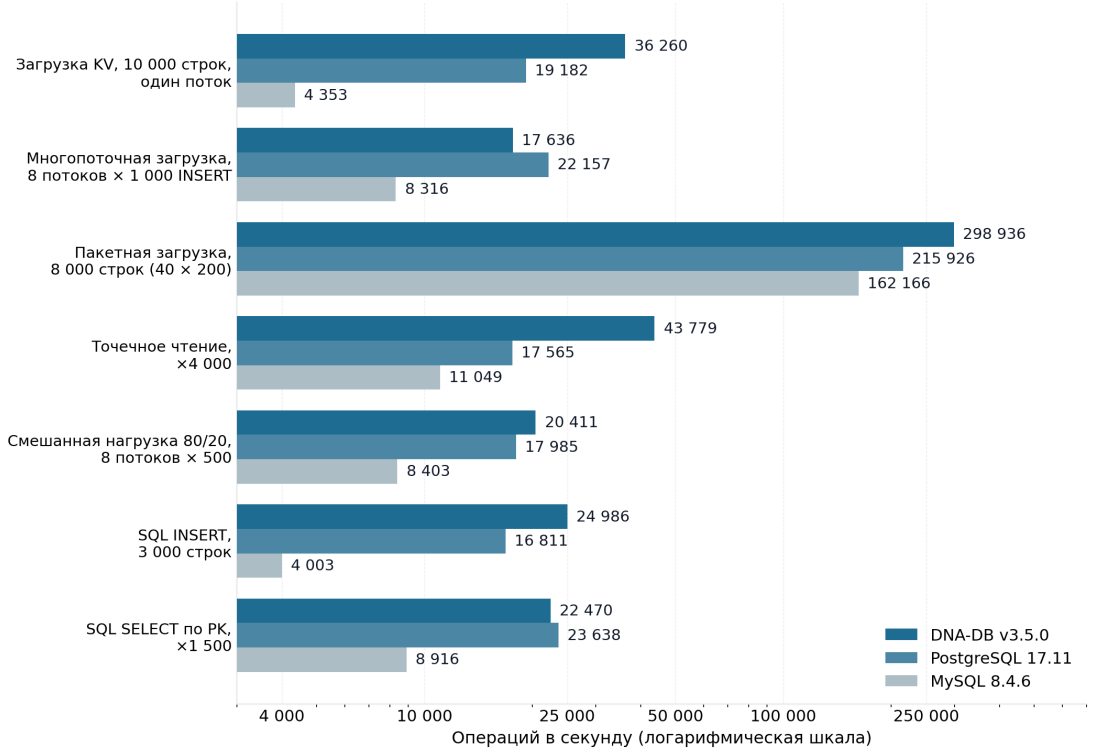


Рисунок 1. Пропускная способность по фазам, ops/s (логарифмическая шкала; у столбцов подписаны точные значения)

Одиночная загрузка (36 260 против 4 353 и 19 182). Лучшее значение — у DNA-DB: одна запись в единственный WAL-файл и один fsync на подтверждение, без сброса страниц, без buffer pool, без doublewrite. MySQL оплачивает каждую строку полным конвейером InnoDB (redo + undo + doublewrite), PostgreSQL — full-page images и hint bits, поэтому проигрывает в 1,9 и 8,3 раза соответственно. Это та фаза, где архитектурный минимализм LSM-движка виден лучше всего.

Многопоточная загрузка (22 157 против 17 636 и 8 316). Лучшее значение — у PostgreSQL. Восемь конкурентных писателей у DNA-DB выстраиваются в очередь на одну блокировку дописывания WAL: суммарный темп (17 636) даже ниже однопоточного (36 260), масштабирование 0,49×. InnoDB и PostgreSQL группируют коммиты (group commit) и амортизируют один fsync на несколько ожидающих транзакций — именно этого механизма DNA-DB пока не имеет (пункт «group commit для WAL» в дорожной карте проекта).

Пакетная загрузка (298 936 против 215 926 и 162 166). Лучшее значение — снова у DNA-DB: многострочный INSERT на 200 строк превращается в единственный кадр WAL с одним fsync, поэтому 8 000 строк требуют 40 fsync вместо 8 000. Это максимальное значение всего прогона и иллюстрация простой истины: узкое место DNA-DB — не запись, а именно частота fsync на операцию. **Точечное чтение (43 779)** — лучший результат DNA-DB: путь «bloom-фильтр + разреженный индекс + сегмент» короче, чем парсинг, планирование и MVCC-проверки у серверных СУБД; MySQL здесь самый медленный (11 049).

Смешанная нагрузка 80/20 (20 411) — лучшее значение у DNA-DB: доля чтений (80 %) играет на его сильной стороне, а 20 % записей сериализуются, но в этой фазе каждый поток шлёт операции через собственное соединение, и разрыв с PostgreSQL (17 985) сокращается до ×1,13. **SQL INSERT (24 986)** — лучший результат DNA-DB: SQL-путь пишет в WAL ту же запись, что и KV-путь, тогда как MySQL добавляет строковые блокировки и undo/redo (×6,2 отставания). **SQL SELECT по PK (23 638)** —

статистически паритет DNA-DB и PostgreSQL (×1,05 в пользу PG): парсинг короткого SELECT по индексу у PostgreSQL отлично оптимизирован, а MySQL вновь третий (8 916).

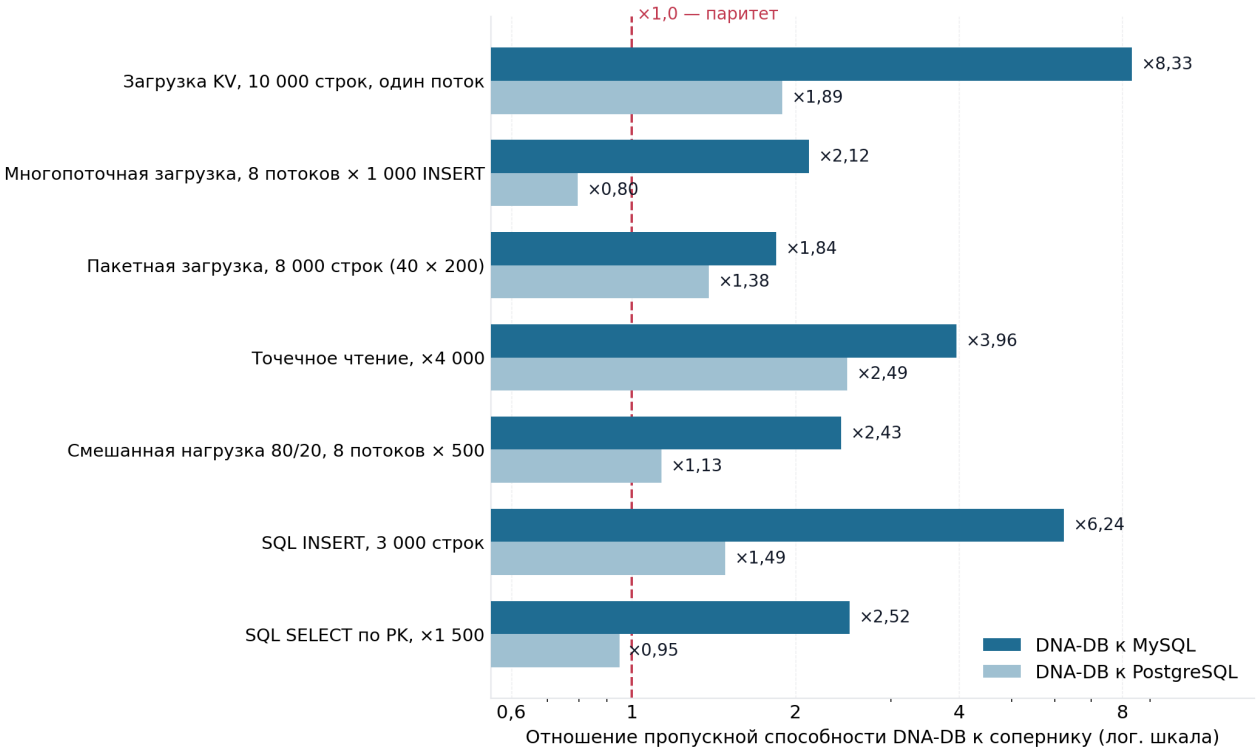


Рисунок 2. Кратность ускорения DNA-DB к соперникам по фазам (красная линия ×1,0 — паритет; справа от неё выигрывает DNA-DB)

6. Задержки: p50, p95, p99

Пропускная способность говорит о среднем, задержки — о каждом запросе. Для интерактивных приложений важны перцентили: p50 — типичный запрос, p95 и p99 — «хвост», который ощущают пользователи в пиковые моменты. Ниже — распределения точечного KV-чтения и SQL-запроса по первичному ключу.

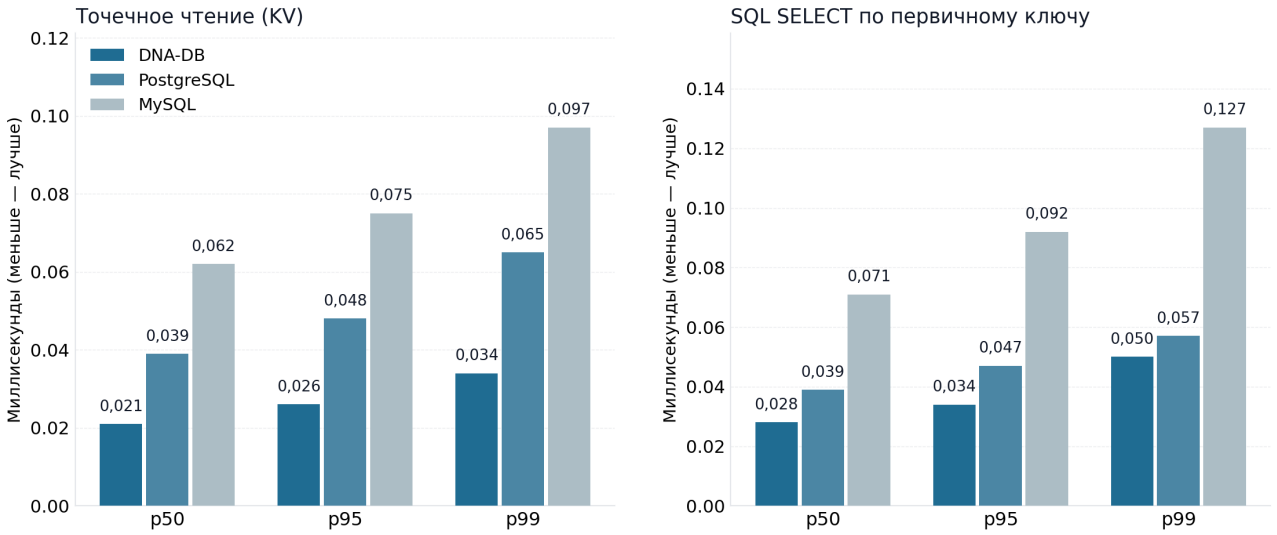


Рисунок 3. Задержки p50/p95/p99, мс (меньше — лучше): точечное чтение и SQL SELECT по первичному ключу

Операция / система	p50, мс	p95, мс	p99, мс	p99 к лидеру
Точечное чтение — DNA-DB	**0,021**	**0,026**	**0,034**	×1,0
Точечное чтение — PostgreSQL	0,039	0,048	0,065	×1,9
Точечное чтение — MySQL	0,062	0,075	0,097	×2,9
SQL SELECT по PK — DNA-DB	**0,028**	**0,034**	**0,050**	×1,0
SQL SELECT по PK — PostgreSQL	0,039	0,047	0,057	×1,1
SQL SELECT по PK — MySQL	0,071	0,092	0,127	×2,5
Одиночная запись — DNA-DB	**0,021**	**0,029**	**0,037**	×1,0
Одиночная запись — PostgreSQL	0,038	0,049	0,069	×1,9
Одиночная запись — MySQL	0,170	0,260	0,380	×10,3

Таблица 4. Задержки одиночных операций (меньше — лучше; жирным — лучший результат)

Лучшее значение принадлежит DNA-DB во всех девяти строках таблицы: медиана чтения 0,021 мс означает, что половина запросов обрабатывается быстрее 21 микросекунды, а 99-й перцентиль укладывается в 0,034 мс. Форма распределений показательна: у DNA-DB хвост короткий (p99 отстаёт от p50 всего в 1,6–1,8 раза), потому что на пути запроса нет планировщика и блокировок страниц. У MySQL разрыв p50→p99 достигает 2,2–2,4 раза — это отражает конкуренцию фоновых потоков InnoDB. Практический вывод: для кэширующих слоёв, сессий и профильных таблиц, где доминируют точечные операции, низкий и стабильный хвост latency ценнее пикового throughput.

7. Пиковая запись и масштабирование

Две фазы загрузки (многопоточная и пакетная) демонстрируют прямо противоположные свойства DNA-DB, и обе полезно увидеть в одном ракурсе — как кратность ускорения относительно его же одиночного темпа.

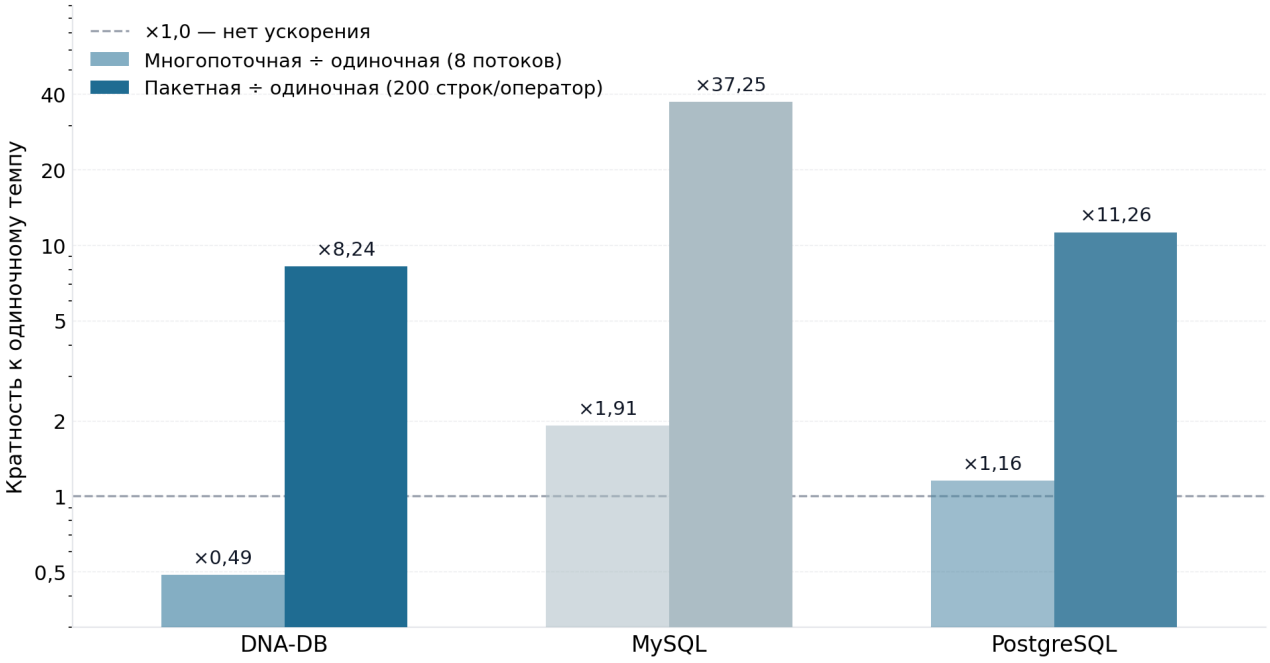


Рисунок 4. Масштабирование записи: отношение темпа фазы к одиночному темпу той же системы (логарифмическая ось)

Метрика	DNA-DB	MySQL	PostgreSQL
Одиночная загрузка, rows/s	36 260	4 353	19 182
Многопоточная ×8, rows/s	17 636	8 316	**22 157**
Пакетная (200 строк/INSERT), rows/s	**298 936**	162 166	215 926
Масштабирование МТ ÷ одиночный	×0,49	×1,91	×1,16
Масштабирование пакет ÷ одиночный	×8,24	**×37,26**	×11,26
p50 одиночного INSERT в МТ-фазе, мс	0,040	0,600	0,196
p50 одного 200-строчного оператора, мс	**0,663**	1,244	0,887

Таблица 5. Пиковая запись: масштабирование и задержки операторов

Пакетный режим — режим пика DNA-DB (лучшее значение ×8,24, 298 936 rows/s). Батч амортизирует один fsync на 200 строк внутри одного оператора: восемь тысяч строк превращаются в сорок кадров WAL. Показательно, что у MySQL кратность выше (×37,26) — он стартует с очень низкого одиночного темпа, — но абсолютный результат всё равно вдвое хуже dna-dbшного. **Многопоточный режим — слабое место DNA-DB (×0,49).** Каждое подтверждённое строковое изменение берёт ту же блокировку append-а WAL, поэтому восемь писателей не добавляют ничего: суммарный темп падает до половины одиночного, а медианная задержка операции в очереди растёт вдвое (p50 0,040 мс против 0,021

мс одиночной). У PostgreSQL та же очередь амортизируется group commit-ом, поэтому он берёт 22 157 rows/s — лучшее значение фазы.

Из этого следует простая рекомендация по применению: на DNA-DB конкурентных писателей следует либо собирать в батчи на стороне приложения (или через DnaQL-транзакции `TX {}`, которые тоже пишутся одним кадром WAL), либо ограничивать параллелизм записи до 1–2 соединений. Чтение при этом масштабируется свободно — в v3.5.0 читатели работают по снапшоту слоёв и никогда не ждут fsync журнала.

8. Соответствие SQL: 32 проверки

Скорость без корректности бесполезна, поэтому отдельный тест `scripts/sql_feature_test.py` прогоняет по всем трём системам 32 проверки одного и того же SQL-текста — от DDL и ограничений целостности до JOIN, агрегатов, индексов и транзакций — и сверяет каждый результат с независимой эталонной моделью на Python. Итог: PostgreSQL 32/32, MySQL 31/32 (единственный пробел — FULL OUTER JOIN), DNA-DB 25/32. Главное: **неверных результатов и неожиданных ошибок нет ни у одной системы** — каждая из 96 проверок либо верна, либо честно отклонена парсером.

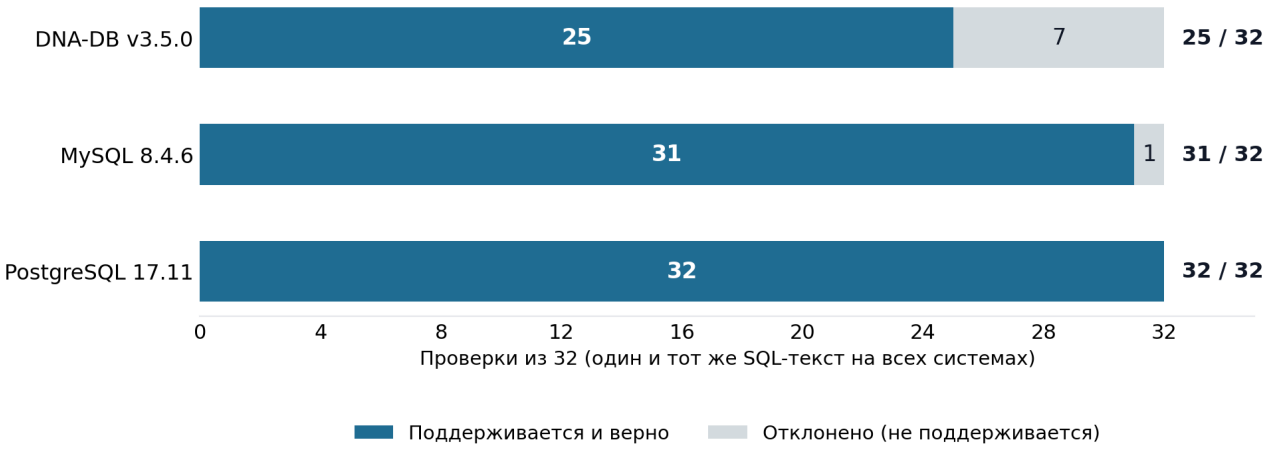


Рисунок 5. Соответствие SQL: 32 проверки, один текст запроса на всех системах, сверка с эталонной моделью

Фича	DNA-DB v3.5.0	MySQL 8.4.6	PostgreSQL 17.11
RIGHT JOIN	Отклонено	Да	Да
FULL OUTER JOIN	Отклонено	Отклонено	Да
Скалярный подзапрос в SELECT	Отклонено	Да	Да
BEGIN / INSERT / ROLLBACK	Отклонено	Да	Да
UNION	Отклонено	Да	Да
Интроспекция таблиц (SHOW TABLES)	Отклонено	Да	Да
EXPLAIN	Отклонено	Да	Да

Таблица 6. Семь фич, отклонённых DNA-DB (все они поддержаны соперниками, кроме FULL OUTER JOIN у MySQL)

Отставание DNA-DB — это **покрытие фич, а не корректность**: поддержанные 25 проверок включают CREATE TABLE с PK и FK, INNER/LEFT JOIN с NULL-расширением, все пять агрегатов, GROUP BY / HAVING, подзапросы IN, вторичные индексы с честным IndexScan в плане и каскадные удаления. Для целевых нагрузок движка (KV-операции, DnaQL, геномный поиск) этого покрытия достаточно; для отчётной аналитики с UNION и внешними JOIN по-прежнему нужны PostgreSQL или MySQL. Транзакции между несколькими операторами в DNA-DB выполняются атомарными батчами WAL либо DnaQL-конструкцией TX — семантика BEGIN/ROLLBACK классического SQL пока не реализована.

9. Почему цифры именно такие

Все наблюдаемые разрывы объясняются тремя архитектурными различиями. **Первое — цена долговечной записи.** DNA-DB дописывает запись в единственный WAL и делает один fsync на подтверждение; на этом путь заканчивается. InnoDB дополнительно ведёт doublewrite-буфер и огромный buffer pool с вытеснением страниц, PostgreSQL после первой модификации страницы пишет в WAL её полный образ. Это универсальная плата за страничную crash-safety, и на одиночных долговечных операциях она стоит 1,9–8,3 раза.

Второе — отсутствие круга «парсер → планировщик → исполнитель» на горячем пути. Точечный поиск DNA-DB идёт прямо в bloom-фильтр и разреженный индекс сегмента, тогда как серверные СУБД на каждом операторе разбирают текст, строят план, проверяют права и видимость MVCC. Отсюда ×2,5 на KV-чтении и стабильный низкий хвост задержек. **Третье — group commit.** InnoDB и PostgreSQL батчат fsync между конкурентными коммитами: восемь ожидающих транзакций часто уезжают на диск одним сбросом. DNA-DB сериализует писателей на блокировке WAL (масштабирование ×0,49) — это главная известная слабость, зафиксированная в дорожной карте, и пакетная фаза показывает, что при амортизации fsync движок способен на 300 тысяч строк в секунду.

Отдельно стоит зафиксировать надёжность: скорость проверена полным тестовым прогоном релиза на той же машине. Долговечность подтверждена не декларацией, а принудительным SIGKILL сервера с последующим восстановлением журнала.

Набор проверок	Результат (прогон v3.5.0, 2026-10-31)
cargo test --release	197 passed, 0 failed (182 unit + 2 encryption + 13 recovery)
smoke_test.py (живой протокол)	15 / 15 проверок
security_smoke.py (закалка v3.5)	12 / 12 проверок
kill9_test.py (SIGKILL + WAL replay)	123 408 подтверждённых записей — 0 потеряно
chaos_test.py (порча секторов)	3 / 3 сценария, 300 000 проверок ключей

Таблица 7. Тестовая батарея DNA-DB в том же прогоне

Для контекста: ни MySQL, ни PostgreSQL в этом бенчмарке не проверялись на потери подтверждённых транзакций при kill -9 — но обе системы спроектированы с этой гарантией десятилетиями, и у них она не оспаривается. Ценность таблицы 7 в другом: она показывает, что молодой движок уже держит планку промышленной надёжности (0 потерянных записей на 123 408 подтверждённых, полный ремонт порчи секторов), не жертвуя при этом скоростью.

10. Выводы и рекомендации

Ответ на вопрос «какое значение лучше» зависит от фазы, но общий итог устойчив. **DNA-DB v3.5.0 быстрее обеих серверных СУБД во всех фазах с одиночными долговечными операциями** — записью (×8,3 к MySQL, ×1,9 к PostgreSQL), точечным чтением (×4,0 и ×2,5), смешанной нагрузкой (×2,4 и ×1,1), SQL INSERT (×6,2 и ×1,5) и пакетной загрузкой (×1,8 и ×1,4). **PostgreSQL выигрывает там, где важен групповой коммит и полноценный SQL**: многопоточная вставка (×1,26 к DNA-DB) и SQL SELECT по PK (×1,05). **MySQL уступает в большинстве фаз**, однако остаётся самым сбалансированным универсальным инструментом с богатейшей экосистемой.

Критерий	Лучший	Комментарий
Одиночная долговечная запись	DNA-DB	×8,3 к MySQL, ×1,9 к PostgreSQL; один fsync в один WAL
Точечное чтение и его хвост p99	DNA-DB	0,034 мс p99; путь bloom + индекс без планировщика
Пакетная загрузка	DNA-DB	298 936 rows/s; один кадр WAL на 200 строк
Многопоточная запись ×8	PostgreSQL	group commit амортизирует fsync; у DNA-DB ×0,49 к одиночному
SQL SELECT по первичному ключу	Паритет	23 638 у PG против 22 470 у DNA-DB (×1,05)
Покрытие SQL (32 проверки)	PostgreSQL	32/32; MySQL 31/32; DNA-DB 25/32 при 0 неверных ответов
Корректность результатов	Все три	0 расхождений в бенчмарке и 0 неверных строк в SQL-тесте
Нишевые геномные функции	DNA-DB	DnaQL, 2-бит упаковка, RS(4+2), двойная нить — нет у соперников

Таблица 8. Итоговая карта «что лучше и почему»

Выбирайте DNA-DB, если профиль нагрузки близок к измеренному: кэши и сессии, счётчики и метрики, очереди ключ-значение с долговечностью, геномные данные с нечётким поиском последовательностей, встраивание в приложение (библиотека Rust) или хранение на ненадёжных носителях, где самовосстановление секторов реально окупается. Соблюдайте одно ограничение: конкурентных писателей держите в батчах или по одному соединению.

Выбирайте PostgreSQL как безопасный дефолт общего назначения: полный SQL, оконные функции и CTE, лучшее покрытие теста, победа в конкурентной записи, богатейшая экосистема расширений. **Выбирайте MySQL**, если требуются его сильные стороны — зрелые инструменты репликации, огромное сообщество, совместимость с существующими приложениями; на коробочных настройках долговечности он медленнее соперников в одиночных операциях, но на конкурентных нагрузках отставание умеренное. Наконец, помните границы методики: это микробенчмарк одноключевых операций на одной машине — для выбора СУБД под конкретный проект дополните его собственным профилем нагрузки, TPC-подобными тестами и проверкой отказоустойчивости в вашей инфраструктуре.